

IsaacLab-Arena Workshop

Participant guide: Simulation Workflow → Setup: Isaac Lab-Arena, on NVIDIA Brev

Tanna TechBiz Physical AI Lab · September 2026 · Arena release/0.2.1, Isaac Sim 6.0.0, Isaac Lab 3.0.0

Fill in before the session

Launchable / instance link: _____

Desktop (noVNC) link: _____

VNC password: _____ Brev login email: _____

1. What you will do today

This module follows NVIDIA's *GR00T End-to-End Workflow* → *Simulation Workflow* → *Setup: Isaac Lab-Arena* page. By the end you will have:

1. A GPU cloud machine with IsaacLab-Arena, Isaac Sim 6.0 and Isaac Lab 3.0 running inside Docker.
2. The Unitree G1 "static apple to plate" environment validated with automated tests.
3. The Isaac Sim GUI open in your browser, running an Arena environment you can poke at.
4. The standalone Isaac GR00T checkout ready for the later fine-tuning and evaluation modules.

Your machine was prepared by a setup script, so steps that normally take an hour (cloning, building the container) are already done. You will verify each one instead of waiting for it.

2. Get to your desktop (5 min)

2.1 If you deploy the Launchable yourself

1. Open the Launchable link from your organizer and sign in to Brev.
2. Click **Deploy Launchable**. Enter a **VNC_PASSWORD** (write it in the box above). Leave the other parameters at their defaults. Keep the suggested GPU.
3. Wait for the instance to show **Running** and **Built**. The first build takes 60-90 minutes, which is why organizers usually deploy the day before.

2.2 If your organizer gave you an instance

1. Open the desktop link they sent. Skip to 2.3.

2.3 Open the browser desktop

1. On the instance page, **Access** tab, click the Secure Link named **desktop** (port 6080). Sign in with your Brev account if asked.
2. If you were given a direct `https://<ip>:6080/vnc.html` link instead, your browser will warn about

- a self-signed certificate. Click **Advanced** → **Proceed**. This is expected.
3. Enter the VNC password. An XFCE Linux desktop appears.
 4. Open a terminal: right-click the desktop → **Open Terminal Here**, or **Applications** → **Terminal Emulator**.

The desktop is rendered on the cloud GPU and streamed to your browser. If it looks small, use the noVNC side panel (left edge) → Settings → Scaling Mode: **Local Scaling**. Fullscreen (F11) helps.

3. Setup Isaac Lab-Arena (workflow steps 1-6)

Commands are typed in the desktop terminal. Everything is already in place; each step tells you how to confirm it.

Step 1-2. The repository

The workflow clones IsaacLab-Arena with submodules. On your machine this was done with the HTTPS URL, so no GitHub SSH key is needed:

```
cd ~/IsaacLab-Arena
git branch --show-current      # expect: release/0.2.1
git submodule status          # two lines, each starting with a commit hash (no
leading "-")
```

If you ever need to redo it yourself (for example on your own computer):

```
cd ~/
git config --global url."https://github.com/".insteadOf "git@github.com:"
git clone --branch release/0.2.1 --recurse-submodules
https://github.com/isaac-sim/IsaacLab-Arena.git
```

Step 3. Host directories

These are mounted into the container as `/datasets`, `/models` and `/eval`:

```
mkdir -p $HOME/datasets $HOME/models $HOME/eval
ls -d $HOME/datasets $HOME/models $HOME/eval
```

Step 4. Launch the Docker container

```
cd ~/IsaacLab-Arena
./docker/run_docker.sh
```

Because the image `isaacsim/isaacsim_arena:latest` is already built, this drops you into a shell inside the container within a few seconds. The prompt changes and your working directory is `/workspaces/isaacsim_arena`.

```
nvidia-smi | head -15          # the GPU is visible inside the container
echo $DISPLAY                  # :0 (the container can open windows on your desktop)
type python                    # alias for /isaac-sim/python.sh, Isaac Sim's own
interpreter
```

Everything from here until section 5 runs **inside the container**. If you close the terminal, run `./docker/run_docker.sh` again; it re-attaches to the same container.

Step 5 (optional). Full test suite

The workflow lists a 20-30 minute verification. Skip it during the live session; run it later if you like:

```
pytest -sv -m "with_cameras and not with_subprocess" isaacsim_arena/tests/
pytest -sv -m "not with_cameras and not with_subprocess" isaacsim_arena/tests/
pytest -sv -m with_subprocess isaacsim_arena/tests/
```

Step 6. Environment variables for the tutorial

```
export DATASET_DIR=/datasets/isaacsim_arena/static_apple_tutorial
mkdir -p $DATASET_DIR
export MODELS_DIR=/models/isaacsim_arena/static_apple_tutorial
mkdir -p $MODELS_DIR
```

4. Validate the environment with automated tests (workflow steps 7-8)

This runs the G1 static pick-and-place environment headlessly and checks scene, task and termination logic. About 2 minutes.

```
python -m pytest isaacsim_arena/tests/test_g1_static_pick_and_place.py -v
```

Expected ending:

```
===== 2 passed, 3488 warnings in 95.54s (0:01:35) =====
```

If the apple falls through the shelf on the very first run, simply run the command again. NVIDIA documents this as a first-execution warm-up effect.

Bonus: see the simulation in the GUI

Still inside the container, run the policy runner with the Kit visualizer. Runner options (`--policy_type`, `--viz`, `--num_steps`) go **before** the environment name; environment options go after it. The first launch takes 3-5 minutes (shader compile and asset download), later ones about a minute. An Isaac Lab window opens on your browser desktop:

```
python isaacsim_arena/evaluation/policy_runner.py \
```

```
--policy_type zero_action --viz kit --num_steps 3000 \
pick_and_place_maple_table \
--embodiment droid_rel_joint_pos \
--pick_up_object rubiks_cube_hot3d_robotlab \
--destination_location bowl_ycb_robotlab \
--hdr home_office_robotlab
```

- Orbit with the right mouse button, pan with the middle button, zoom with the wheel.
- Hold **Shift** and left-drag an object to nudge it.
- Try swapping `--hdr home_office_robotlab` for `--hdr billiard_hall_robotlab`, or the cube for `mustard_bottle_hot3d_robotlab`.
- The `zero_action` policy holds the robot still; the point is to see the scene. Steps run at a few per second with the GUI on, so 3000 steps is roughly 10-15 minutes. Press **Ctrl-C** in the terminal to stop earlier.
- Omit `--viz kit` to run the same thing headless, which is what the automated tests do.

5. Setup Isaac GR00T outside the container (workflow steps 9-12)

Open a **second terminal** on the desktop (not inside Docker). The workflow keeps GR00T separate so it can be updated without rebuilding the Arena container. The setup script has already cloned it at the pinned commit and created the `uv` environment; verify:

```
export ISAAC_GR00T_DIR=$HOME/Isaac-GR00T
cd $ISAAC_GR00T_DIR
git rev-parse HEAD          # expect 4b1dca9d88d2a0b9ea5a65aa61c82ff89f5c4f0e
ls .venv && uv --version    # environment exists
```

If either check fails, do it by hand exactly as the workflow says:

```
git clone https://github.com/NVIDIA/Isaac-GR00T.git $ISAAC_GR00T_DIR
cd $ISAAC_GR00T_DIR
git checkout 4b1dca9d88d2a0b9ea5a65aa61c82ff89f5c4f0e
uv sync
```

Checkpoint reached: Arena container running, environment tests passing, GUI verified, GR00T environment ready. This is the state the next workflow pages assume.

6. What comes next in the workflow

#	Page	What it covers
3	Sim Environment Code Review	Reads <code>galileo_g1_static_pick_and_place_environment.py</code> : assets, embodiment, scene and task logic.

4	Sim Teleop and WBC	OpenXR teleoperation with whole-body control to collect demonstrations.
5	Sim Data Export	Exporting the collected episodes to <code>\$DATASET_DIR</code> in LeRobot format.
6	GR00T Fine-Tuning on Sim Data	Post-training GR00T 1.7 in <code>~/Isaac-GR00T</code> on the exported data.
7	Sim Evaluation	Closed-loop evaluation of the fine-tuned checkpoint through Arena's remote-policy server/client.

All pages: <https://docs.nvidia.com/learning/physical-ai/gr00t-e2e-workflow/latest/simulation-workflow/>

7. Troubleshooting

Problem	Try this
Desktop link does not load	Reload once. If still blank, ask the organizer to run <code>sudo systemctl restart gpu-desktop</code> on your instance.
Wrong VNC password	It is the value you (or the organizer) entered as <code>VNC_PASSWORD</code> at deploy time. It is also in <code>~/WORKSHOP.md</code> on the instance.
No window appears for <code>policy_runner</code>	Check <code>--viz</code> <code>kit</code> is present and placed before the environment name. Inside the container run <code>echo \$DISPLAY</code> ; it must print <code>:0</code> . If empty: <code>exit</code> , then <code>export DISPLAY=:0</code> and start <code>./docker/run_docker.sh</code> again.
"the following arguments are required: <code>--policy_type</code> "	Argument order. Runner options go before the environment name, environment options after it.
"Either <code>num_steps</code> or <code>num_episodes</code> must be provided"	Add <code>--num_steps 3000</code> (or <code>--num_episodes 3</code>) before the environment name.
"permission denied" from docker	Log out and back in to the desktop terminal, or run <code>newgrp docker</code> .
Test fails with the apple falling	Run the <code>pytest</code> command a second time.
Container shell is slow to start the first time	Isaac Sim compiles shaders on first launch (2-5 minutes). Subsequent launches are fast.
Browser desktop is laggy	Lower the browser window size, enable Local Scaling, or close other tabs. The simulation itself runs at full speed on the GPU.

8. Quick reference

```
# enter / re-enter the Arena container
cd ~/IsaacLab-Arena && ./docker/run_docker.sh

# inside the container
python -m pytest isaacrab_arena/tests/test_g1_static_pick_and_place.py -v
python isaacrab_arena/evaluation/policy_runner.py --policy_type zero_action --viz kit --
num_steps 3000 \
    pick_and_place_maple_table --embodiment droid_rel_joint_pos --hdr home_office_robotab

# on the host
~/start-desktop.sh          # restart the browser desktop if it dies
cat ~/WORKSHOP.md          # URL and password for this machine
```